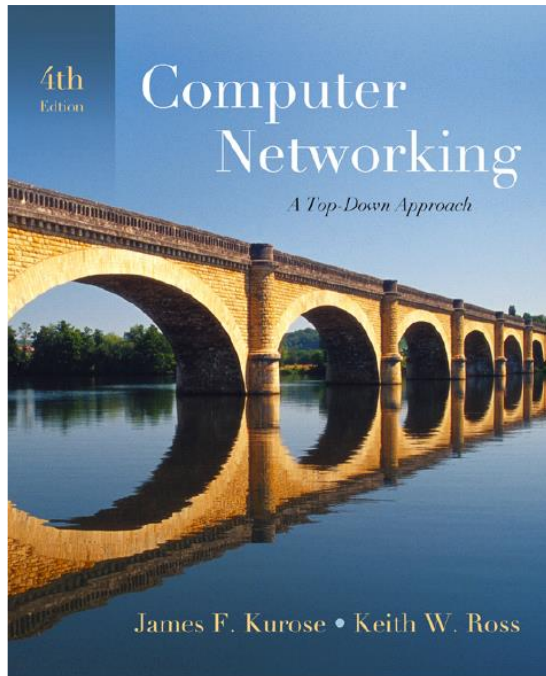


CPE 400 / 600

Computer Communication Networks



Lecture 5

Chapter 2 Application Layer

slides are modified from **J. Kurose & K. Ross**

Chapter 2: Application layer

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Electronic Mail
 - ❖ SMTP, POP3, IMAP
- ❑ 2.5 DNS
- ❑ 2.6 P2P applications
- ❑ 2.7 Socket programming with TCP
- ❑ 2.8 Socket programming with UDP

Processes communicating

Process:

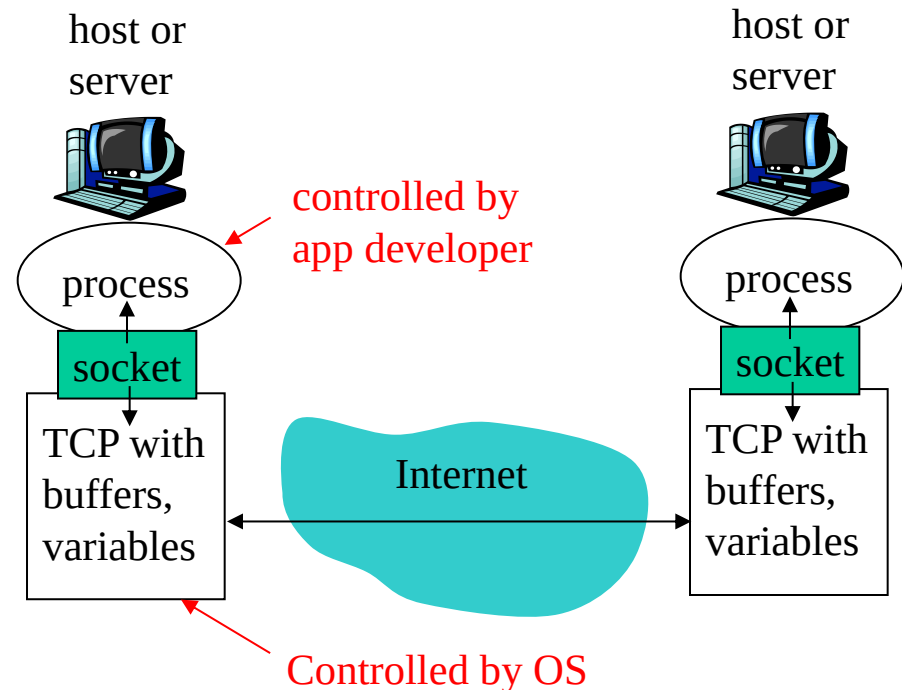
program running within a host

Client process:

initiates communication

Server process:

waits to be contacted



process sends/receives messages to/from its **socket**

identifier includes both **IP address** and **port numbers** associated with process on host.

App-layer protocol defines

- ❑ Types of messages exchanged,
 - ❖ e.g., request, response
- ❑ Message syntax:
 - ❖ what fields in messages & how fields are delineated
- ❑ Message semantics
 - ❖ meaning of information in fields
- ❑ Rules for when and how processes send & respond to messages

Public-domain protocols:

- ◆ defined in RFCs
- ◆ allows for interoperability
- ◆ e.g., HTTP, SMTP

Proprietary protocols:

- ◆ e.g., Skype

Transport service requirements of common apps

Application	Data loss	Throughput	Time Sensitive
file transfer	no loss	elastic	no
e-mail	no loss	elastic	no
Web documents	no loss	elastic	no
real-time audio/video	loss-tolerant	audio: 5kbps-1Mbps video:10kbps-5Mbps	yes, 100's msec
stored audio/video	loss-tolerant	same as above	yes, few secs
interactive games	loss-tolerant	few kbps up	yes, 100's msec
instant messaging	no loss	elastic	yes and no

Internet transport protocols services

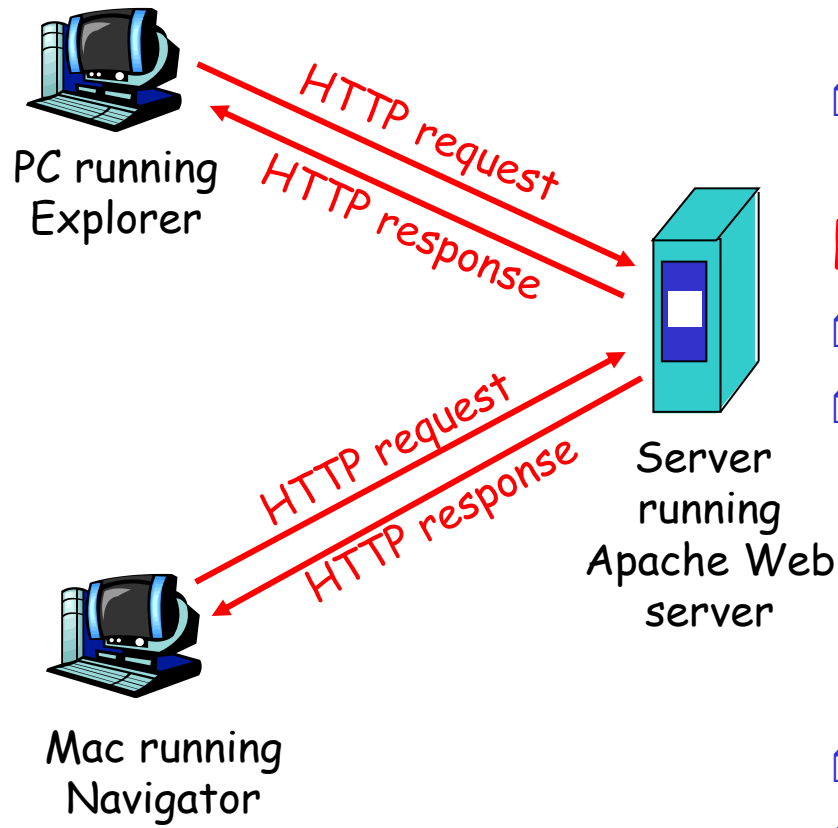
TCP service:

- ❑ *connection-oriented*: setup required between client and server processes
- ❑ *reliable transport* between sending and receiving process
- ❑ *flow control*: sender won't overwhelm receiver
- ❑ *congestion control*: throttle sender when network overloaded
- ❑ *does not provide*: timing, minimum throughput guarantees, security

UDP service:

- ❑ unreliable data transfer between sending and receiving process
- ❑ does not provide: connection setup, reliability, flow control, congestion control, timing, throughput guarantee, or security

HTTP overview



- ❑ Web page consists of **base HTML-file** which includes several referenced **objects**
- ❑ Each object is addressable by a **URL**

HTTP: hypertext transfer protocol

- ❑ Web's application layer protocol
- ❑ client/server model
 - ❖ **client**: browser that requests, receives, "displays" Web objects
 - ❖ **server**: Web server sends objects in response to requests
- ❑ uses TCP
- ❑ is **"stateless"**

HTTP connections

Nonpersistent HTTP

- ❑ At most one object is sent over a TCP connection.

Persistent HTTP

- ❑ Multiple objects can be sent over single TCP connection between client and server.

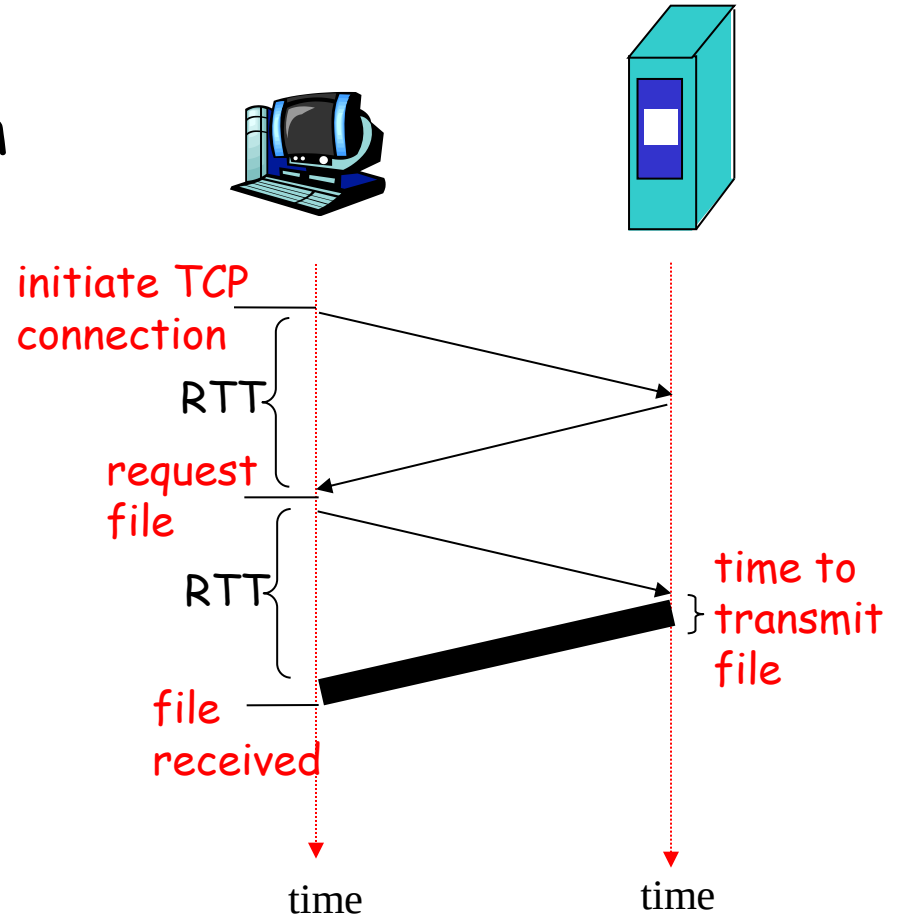
Non-Persistent HTTP: Response time

Definition of RTT: time for a small packet to travel from client to server and back.

Response time:

- one RTT to initiate TCP connection
- one RTT for HTTP request and first few bytes of HTTP response to return
- file transmission time

total = $2RTT + \text{transmit time}$



Persistent HTTP

Nonpersistent HTTP issues:

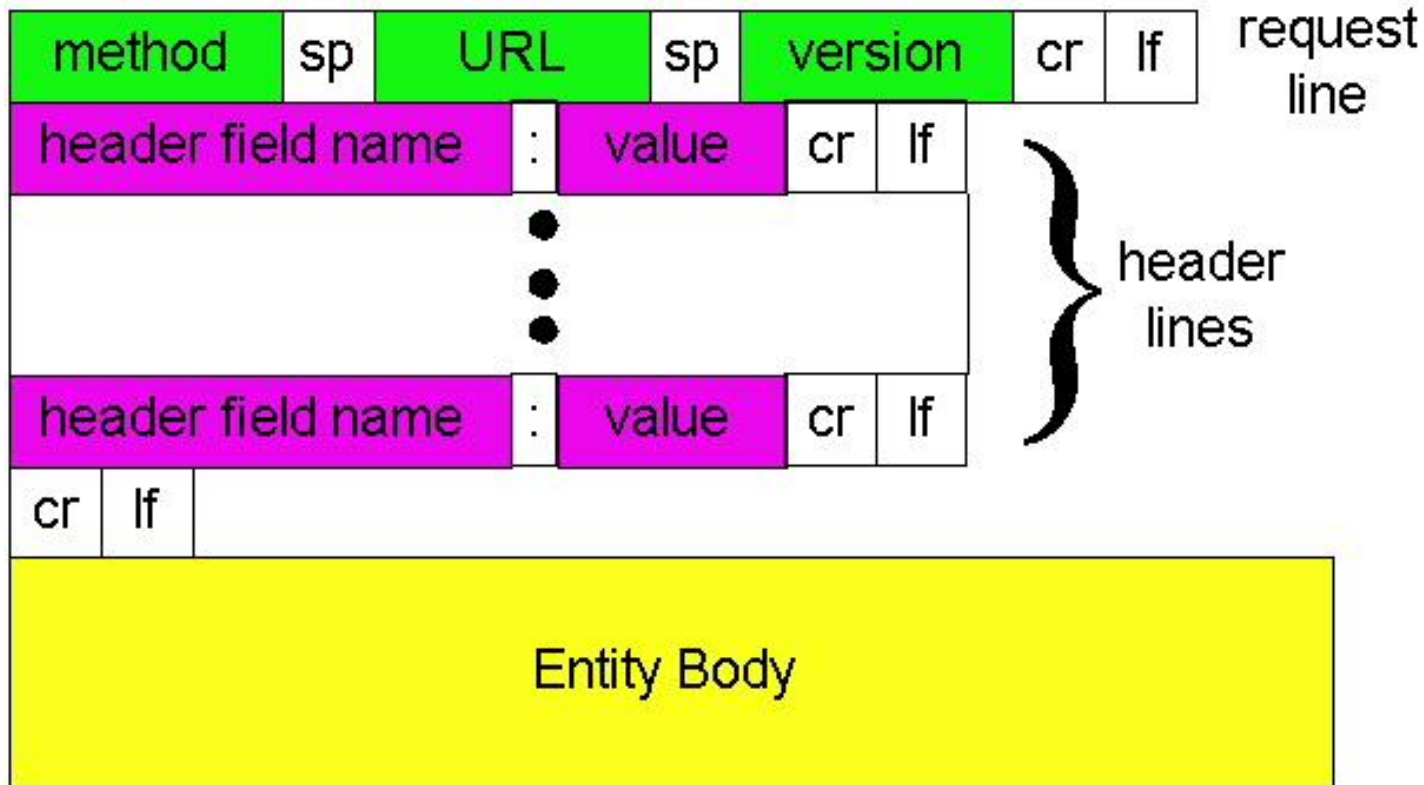
- ❑ requires 2 RTTs per object
- ❑ OS overhead for *each* TCP connection
- ❑ browsers often open parallel TCP connections to fetch referenced objects

Persistent HTTP

- ❑ server leaves connection open after sending response
- ❑ subsequent HTTP messages between same client/server sent over open connection
- ❑ client sends requests as soon as it encounters a referenced object
- ❑ as little as one RTT for all the referenced objects

HTTP messages

- two types of HTTP messages: *request, response*
- *HTTP request message:*
 - ❖ ASCII (human-readable format)



Method types

HTTP/1.0

□ GET

- ❖ request an object from server

□ POST

- ❖ upload information using forms

□ HEAD

- ❖ asks server to leave requested object out of response

HTTP/1.1

□ GET, POST, HEAD

□ PUT

- ❖ uploads file in entity body to path specified in URL field

□ DELETE

- ❖ deletes file specified in the URL field

Cookies: Keeping state

What cookies can bring:

- ❑ authorization
- ❑ shopping carts
- ❑ recommendations
- ❑ user session state (Web e-mail)

Cookies and privacy: aside

- ❑ cookies permit sites to learn a lot about you
- ❑ you may supply name and e-mail to sites

How to keep "state":

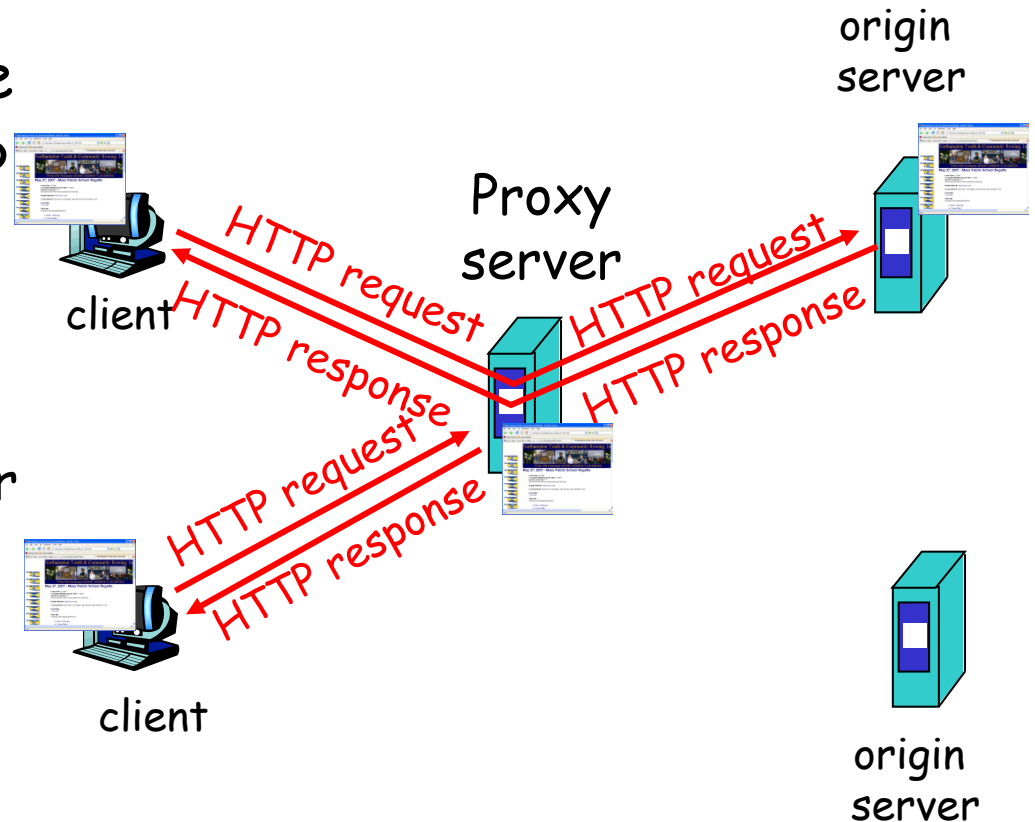
- ❑ protocol endpoints: maintain state at sender/receiver over multiple transactions
- ❑ cookies: http messages carry state

Web caches (proxy server)

Goal: satisfy client request without involving origin server

- ❑ user sets browser: Web accesses via cache
- ❑ browser sends all HTTP requests to cache

- ❑ Why Web caching?
 - ❖ reduce response time for client request
 - ❖ reduce traffic on an institution's access link.
 - ❖ enables "poor" content providers to effectively deliver content

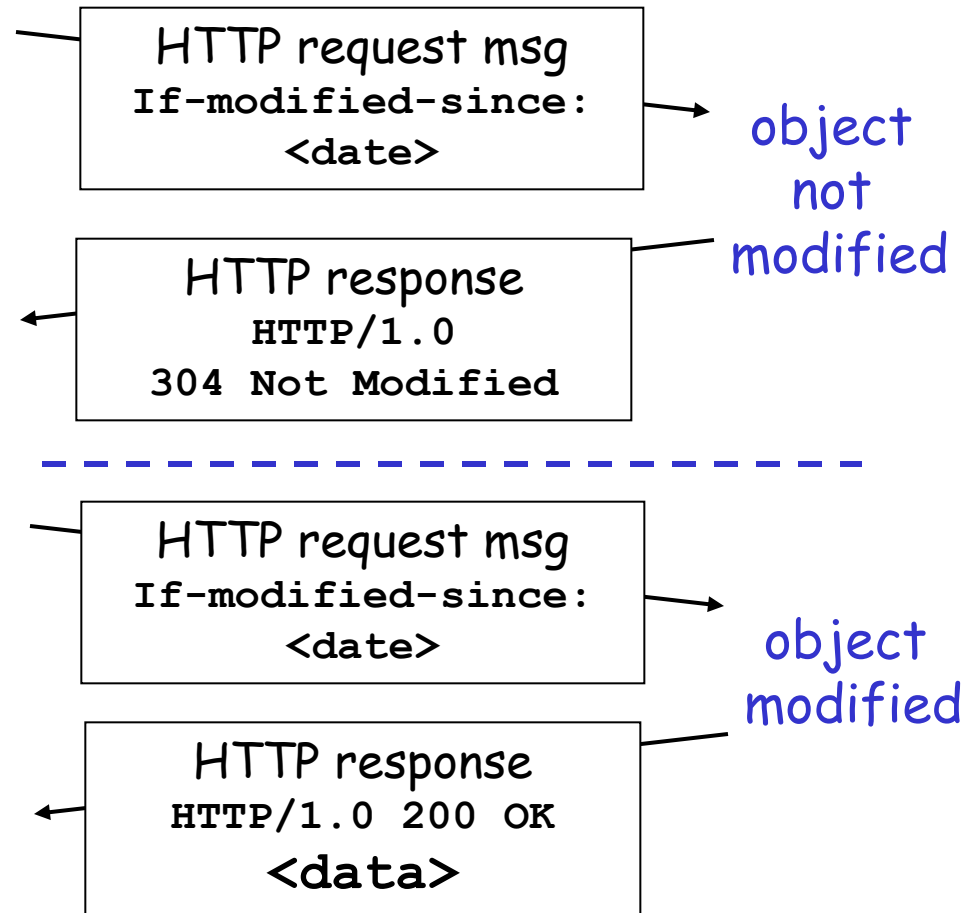


Conditional GET

- **Goal:** don't send object if cache has up-to-date cached version
- **cache:** specify date of cached copy in HTTP request
If-modified-since: <date>
- **server:** response contains no object if cached copy is up-to-date:
HTTP/1.0 304 Not Modified

cache

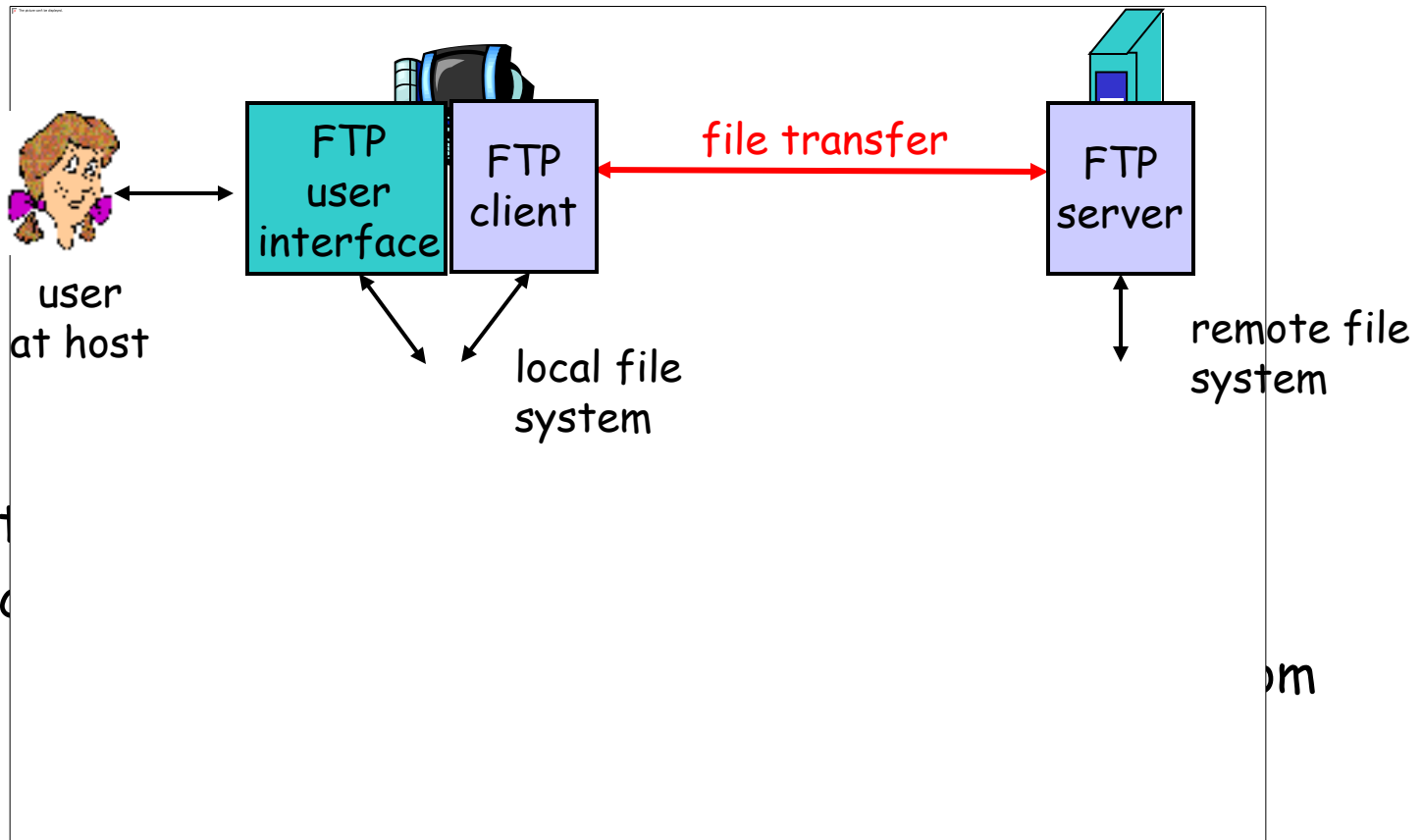
server



Lecture 5: Outline

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Electronic Mail
 - ❖ SMTP, POP3, IMAP
- ❑ 2.5 DNS

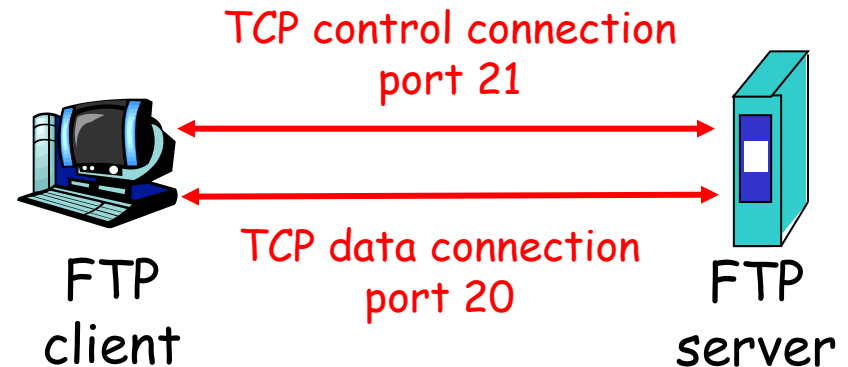
FTP: the file transfer protocol



- ☐ ftp: RFC 959
- ☐ ftp server: port 21

FTP: separate control, data connections

- ❑ FTP client contacts FTP server at port 21
- ❑ client authorized over control connection
- ❑ client browses remote directory by sending commands over control connection.
- ❑ when server receives file transfer command, server opens 2nd TCP connection (for file) to client
- ❑ after transferring one file, server closes data connection.
- ❑ server opens another TCP data connection to transfer another file.
- ❑ control connection: "out of band"
- ❑ FTP server maintains "state": current directory, earlier authentication



FTP commands, responses

Sample commands:

- ❑ sent as ASCII text over control channel
- ❑ USER *username*
- ❑ PASS *password*
- ❑ LIST return list of file in current directory
- ❑ RETR *filename* retrieves (gets) file
- ❑ STOR *filename* stores (puts) file onto remote host

Sample return codes

- ❑ status code and phrase (as in HTTP)
- ❑ 331 Username OK, password required
- ❑ 125 data connection already open; transfer starting
- ❑ 425 Can't open data connection
- ❑ 452 Error writing file

FTP issues

- ❑ Multiple connections are used
 - ❖ for each directory listing and file transmission
- ❑ No integrity check at receiver
- ❑ Messages are sent in clear text
 - ❖ including Passwords and file contents
 - ❖ can be sniffed by eavesdroppers
- ❑ Solution
 - ❖ Secure FTP (SSH FTP)
 - allows a range of operations on remote files
 - ❖ FTPS (FTP over Secure Sockets Layer (SSL))
 - ❖ Transport Layer Security (TLS) encryption

Lecture 5: Outline

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Electronic Mail
 - ❖ SMTP
 - ❖ POP3
 - ❖ IMAP
- ❑ 2.5 DNS

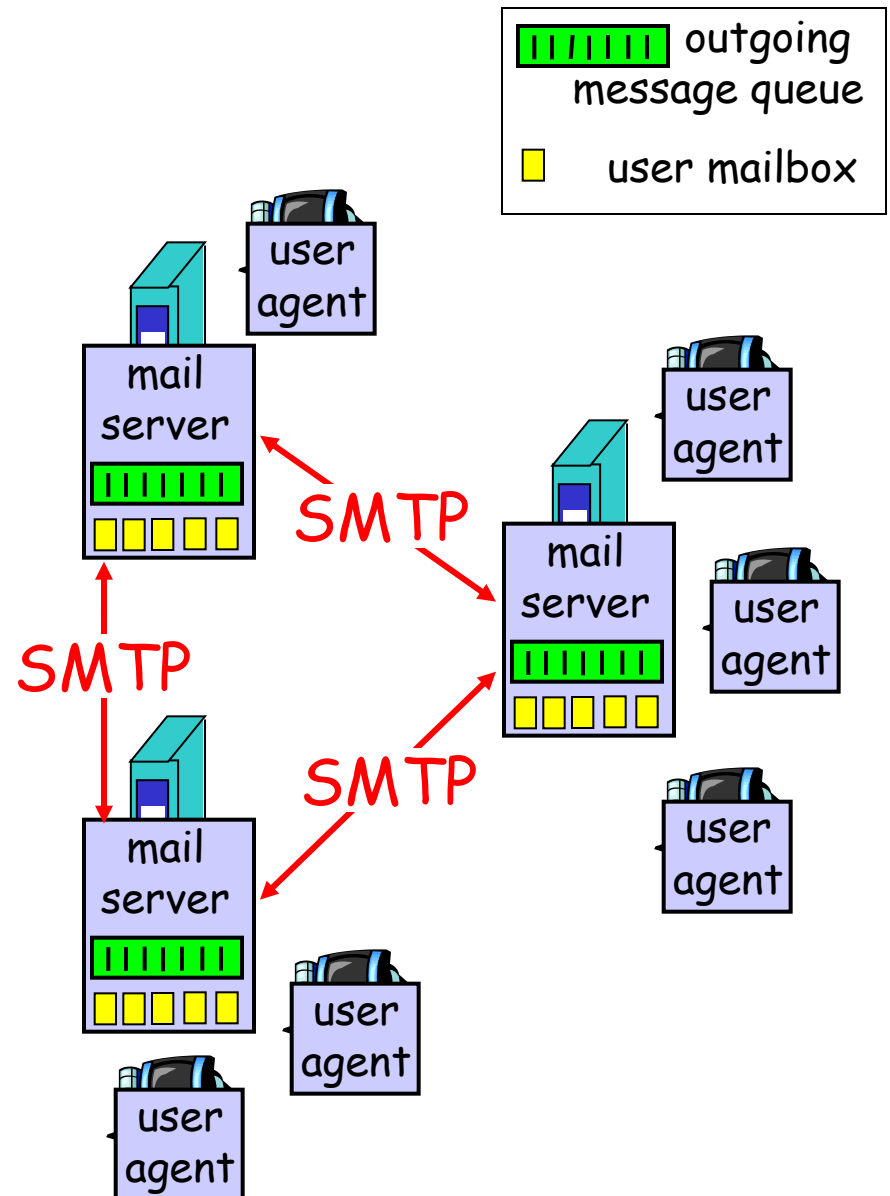
Electronic Mail

Three major components:

- ❑ user agents
- ❑ mail servers
- ❑ simple mail transfer protocol: SMTP

User Agent

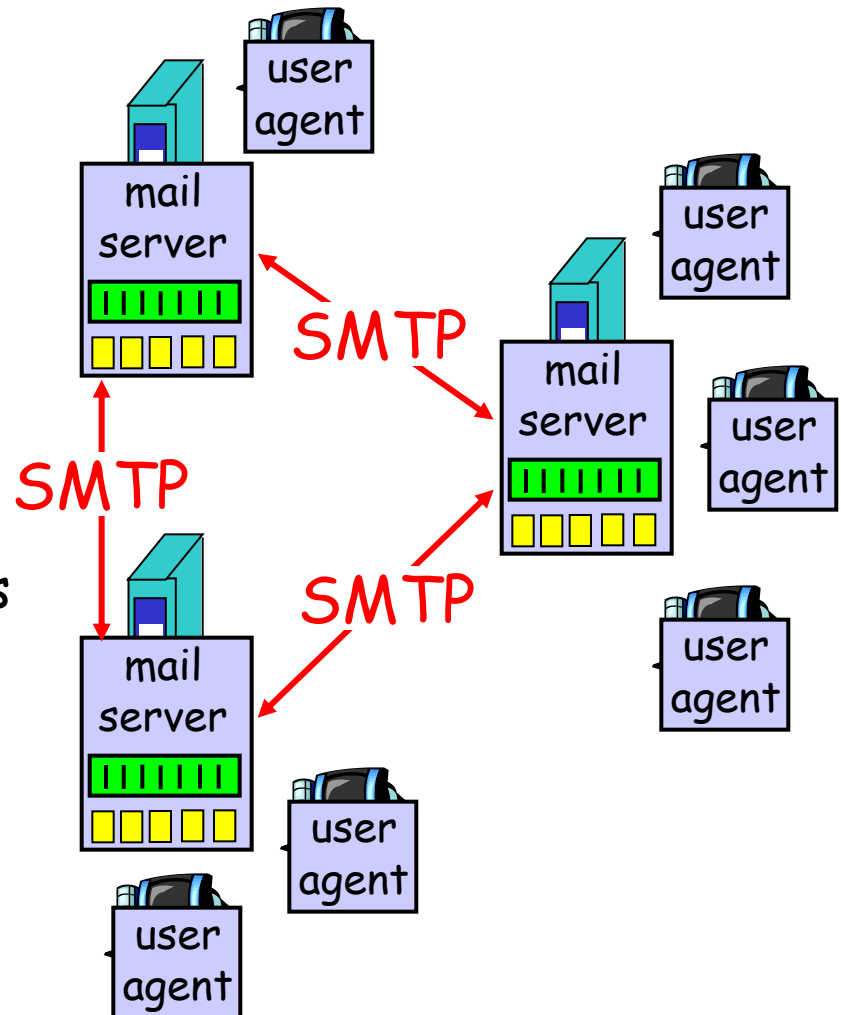
- ❑ a.k.a. "mail reader"
- ❑ composing, editing, reading mail messages
- ❑ e.g., Eudora, Outlook, elm, Mozilla Thunderbird
- ❑ outgoing, incoming messages stored on server



Electronic Mail: mail servers

Mail Servers

- ❑ **mailbox** contains incoming messages for user
- ❑ **message queue** of outgoing (to be sent) mail messages
- ❑ **SMTP protocol** between mail servers to send email messages
 - ❖ client: sending mail server
 - ❖ "server": receiving mail server

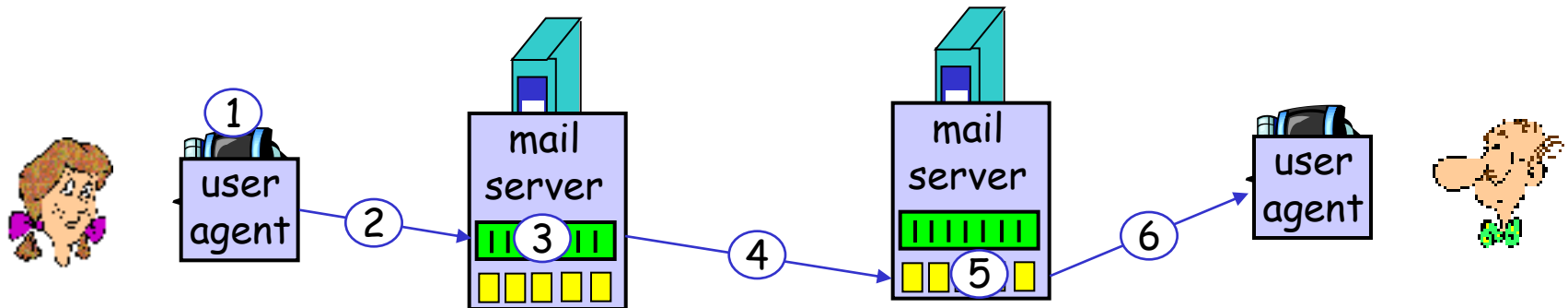


Electronic Mail: SMTP [RFC 2821]

- ❑ uses TCP to reliably transfer email message from client to server (port 25)
- ❑ direct transfer: sending server to receiving server
- ❑ three phases of transfer
 - ❖ handshaking (greeting)
 - ❖ transfer of messages
 - ❖ closure
- ❑ command/response interaction
 - ❖ **commands**: ASCII text
 - ❖ **response**: status code and phrase
- ❑ messages must be in 7-bit ASCII

Scenario: Alice sends message to Bob

- 1) Alice uses UA to compose message and "to" bob@some school . edu
- 2) Alice's UA sends message to her mail server; message placed in message queue
- 3) Client side of SMTP opens TCP connection with Bob's mail server
- 4) SMTP client sends Alice's message over the TCP connection
- 5) Bob's mail server places the message in Bob's mailbox
- 6) Bob invokes his user agent to read message



Sample SMTP interaction

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

SMTP: final words

- ❑ SMTP uses persistent connections
- ❑ SMTP requires message (header & body) to be in 7-bit ASCII
- ❑ SMTP server uses `CRLF.CRLF` to determine end of message

Comparison with HTTP:

- ❑ HTTP: pull
- ❑ SMTP: push
- ❑ both have ASCII command/response interaction, status codes
- ❑ HTTP: each object encapsulated in its own response msg
- ❑ SMTP: multiple objects sent in multipart msg

Mail message format

SMTP: protocol for exchanging email msgs

RFC 822: standard for text message format:

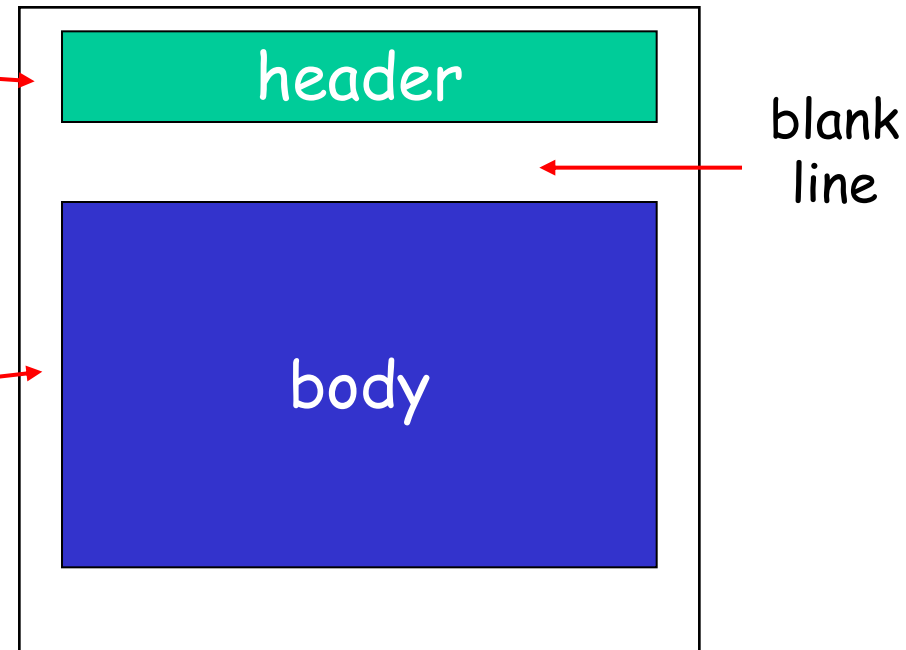
- header lines, e.g.,

- ❖ To:
- ❖ From:
- ❖ Subject:

different from SMTP commands!

- body

- ❖ the "message",
ASCII characters only



Message format: multimedia extensions

- ❑ MIME: multimedia mail extension, RFC 2045, 2056
- ❑ additional lines in msg header declare MIME content type

MIME version

method used
to encode data

multimedia data
type, subtype,

parameter declaration

encoded data

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Picture of yummy crepe.
MIME-Version: 1.0
Content-Transfer-Encoding: base64
Content-Type: image/jpeg

base64 encoded data .....
.....
.....base64 encoded data
```

Mail access protocols



□ SMTP

□ Mail

❖ POP

❖ IMAP

• manipulation of stored msgs on server

❖ HTTP: gmail, Hotmail, Yahoo! Mail, etc.

POP3 protocol

authorization phase

□ client commands:

- ❖ user: declare username
- ❖ pass: password

□ server responses

- ❖ +OK
- ❖ -ERR

transaction phase, client:

- list: list message numbers
- retr: retrieve message by number
- dele: delete
- quit

```
S: +OK POP3 server ready
C: user bob
S: +OK
C: pass hungry
S: +OK user successfully logged on
```

```
C: list
S: 1 498
S: 2 912
S: .
C: retr 1
S: <message 1 contents>
S: .
C: dele 1
C: retr 2
S: <message 1 contents>
S: .
C: dele 2
C: quit
S: +OK POP3 server signing off
```

POP3 (more) and IMAP

More about POP3

- ❑ Previous example uses “download and delete” mode.
- ❑ Bob cannot re-read e-mail if he changes client
- ❑ “Download-and-keep”: copies of messages on different clients
- ❑ POP3 is stateless across sessions

IMAP

- ❑ Keep all messages in one place: the server
- ❑ Allows user to organize messages in folders
- ❑ IMAP keeps user state across sessions:
 - ❖ names of folders and mappings between message IDs and folder name

Try SMTP interaction for yourself:

- ❑ `telnet servername 25`
- ❑ see 220 reply from server
- ❑ enter HELO, MAIL FROM, RCPT TO, DATA, QUIT commands

above lets you send email without using email client (reader)

Lecture 5: Outline

- ❑ 2.1 Principles of network applications
- ❑ 2.2 Web and HTTP
- ❑ 2.3 FTP
- ❑ 2.4 Electronic Mail
 - ❖ SMTP
 - ❖ POP3
 - ❖ IMAP
- ❑ 2.5 DNS

DNS: Domain Name System

People: many identifiers:

- ❖ SSN, name, passport #

Internet hosts, routers:

- ❖ IP address (32 bit) - used for addressing datagrams
- ❖ "name", e.g., ww.yahoo.com - used by humans

Domain Name System:

- ❑ *distributed database* implemented in hierarchy of many *name servers*
- ❑ *application-layer protocol* host, routers, name servers to communicate to *resolve* names (address/name translation)
 - ❖ note: core Internet function, implemented as application-layer protocol
 - ❖ complexity at network's "edge"

DNS services

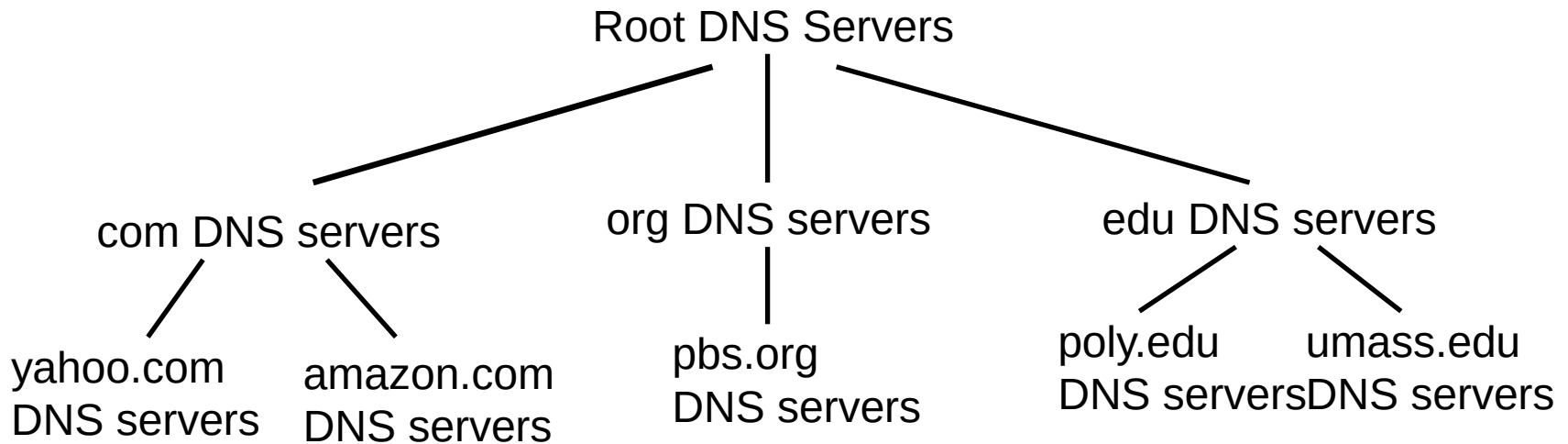
- ❑ hostname to IP address translation
- ❑ host aliasing
 - ❖ Canonical, alias names
- ❑ mail server aliasing
- ❑ load distribution
 - ❖ replicated Web servers: set of IP addresses for one canonical name

Why not centralize DNS?

- ❑ single point of failure
- ❑ traffic volume
- ❑ distant centralized database
- ❑ maintenance

doesn't scale!

Distributed, Hierarchical Database



Client wants IP for www.amazon.com; 1st approx:

- ❑ client queries a **root server** to find **com** DNS server
- ❑ client queries **com** DNS server to get **amazon.com** DNS server
- ❑ client queries **amazon.com** DNS server to get **IP** address for www.amazon.com

Lecture 5: Summary

- ❑ Application
- ❑ Web and HTTP
- ❑ File Transfer Protocol
- ❑ Electronic Mail
 - ❖ SMTP
 - ❖ POP3
 - ❖ IMAP
- ❑ Domain Name Service